

Unit - II

Smart Data Querying with SQL & PL/SQL.

Syllabus

SQL : Data types ; DDL , DML , DCL and TCL commands (Create User, Grant Revoke Queries)

Tables : Create , Insert , Update , Delete , Alter , Drop , SELECT , Joins , Views (Create , Update , Drop) ; Aggregation Functions , set operations , SQL Operators , SQL Predicates , Nested queries.

PL/SQL : Stored ~~proc~~ procedures , functions , triggers , assertions , roles , and privileges.

* SQL - Structured Query Language

- It is a Standard language. Used to retrieve, and manage data in database.
- SQL is especially used with relational database, where data is organized in tables (rows & columns).

* Data Types

- INT / Integer - whole numbers.
e.g. age INT
- VARCHAR(n) - text of variable length.
e.g. name VARCHAR(50)
- CHAR(n) - Fixed-length text.
- DATE - date values
- FLOAT / DECIMAL - decimal numbers.
- BOOLEAN - true / false.

* DDL (Data Definition Language)

DDL commands are used to define or change the structure of database objects.

DDL Commands

1. **CREATE** - Create database objects.

Syntax:

```
CREATE TABLE Student (  
    id INT,  
    name VARCHAR(50)  
);
```

2. **ALTER** - modify table structure

Syntax:

```
ALTER TABLE Student ADD age INT;
```

3. **DROP** - delete database objects

Syntax:

```
DROP TABLE Student;
```

4. **TRUNCATE** - remove all records but keep table structure

Syntax:

```
TRUNCATE TABLE Student;
```


• DML (Data Manipulation Language)

--- DML commands are used to manage data inside tables.

- DML Commands

1. **INSERT** - add new records

Syntax:

```
INSERT INTO Student VALUES (1, 'Ravi', 20);
```

2. **UPDATE** - modify existing data

Syntax:

```
UPDATE Student SET age = 21 WHERE id = 1;
```

3. **DELETE** - remove records

Syntax:

```
DELETE FROM Student WHERE id = 1;
```

* DCL (Data Control Language)

- DCL controls access and permissions.
- primarily for security purpose.

1. **GRANT** - give permissions

Syntax:

GRANT SELECT ON Student To User1;

2. **REVOKE** - remove permissions.

Syntax:

REVOKE SELECT ON Student FROM User1;

* **TCL (Transaction Control Language)**

- TCL manages transactions (a group of SQL statements)
- They are essential for maintaining the ACID properties (Atomicity, Consistency, Isolation, Durability) of a database.

- **TCL Commands.**

1. **COMMIT** - Save changes permanently

Syntax:

COMMIT;

2. **ROLLBACK** - undo changes

Syntax:

ROLLBACK;

3. **SAVEPOINT** - Set a point to rollback to

Syntax: SAVEPOINT savepoint_name;

Example :

SAVEPOINT S1;

ROLLBACK TO S1;

TABLES

- In SQL, a table is a store data in rows and columns.

• Each row = record

• Each column = field

Example :

CREATE TABLE Employee (

emp-id INT,

emp-name VARCHAR(50),

Salary DECIMAL(10,2)

);

Operations on the Table.

1. Create

7. SELECT

2. Insert

8. Joins

3. Update

9. Views

4. Delete

5. Alter

6. Drop

* **SELECT** - Used to retrieve data from table.

example :

```
SELECT * FROM Employee;
SELECT emp-name, salary FROM
Employee WHERE salary > 30000;
```

* **Views** - A view is a virtual table created from a query.

example :

- **Create view**

```
CREATE VIEW HighSalary AS
SELECT emp-name, salary
FROM Employee
WHERE salary > 50000;
```

- **Drop / update view**

```
DROP VIEW HighSalary;
```

* **Joins** - Joins combine data from multiple tables.

- **Types of Joins**

1. **INNER Join** - matching records only.

2. LEFT JOIN - all from left table
3. RIGHT JOIN - all from right table
4. FULL JOIN - all records.

example:

```
SELECT e.emp-name, d.dept-name
FROM Employee e
INNER JOIN Department d
ON e.dept-id = d.dept-id;
```

* Aggregation Functions.

- An Aggregation Function in SQL perform a calculation on a set of values (multiple rows) and return a single value as a result.

• Common Aggregate Functions.

1. **AVG()**: Return the average value of a numeric column.
2. **COUNT()**: Counts the number of rows or non NULL values in a column.
3. **MAX()**: Returns the largest (maximum) value in a column.

4. **MIN()**: Returns the smallest (minimum) value in a column.

5. **SUM()**: Return the total sum of the values in a numeric column.

• Example :

```
SELECT AVG (salary) FROM Employee ;
```

* Set Operations

- It combines the results of two or more SELECT statements into a single result set.

• Types of set operations

1. **UNION**: combines results (no duplicates)

2. **UNION ALL**: Includes duplicates

3. **INTERSECT**: Common records

4. **EXCEPT / MINUS**: difference

example :

```
SELECT name FROM A
```

```
UNION
```

```
SELECT name FROM B;
```

* SQL Operators

Operators perform operations in conditions

Types :

1. Arithmetic Operators

+ : Addition

- : Subtraction

* : Multiplication

/ : Division

% : Modulus

2. Comparison Operator

= : Equal to

> : Greater than

< : Less than

>= : Greater than or equal to

<= : less than or equal to

!= : Not equal to

3. Logical Operator :

AND

OR

NOT

4. Special Operator :

LIKE

IN

BETWEEN

IS NULL

* SQL Predicates

- A predicate is a condition or expression that evaluates to Boolean value (either True, False or Unknown)
- Predicates are primarily used in WHERE, HAVING, and ON clauses of SQL Statement to filter data, determining which rows are included in the result set.

example :

WHERE age BETWEEN 18 AND 25

WHERE name LIKE 'A%'

WHERE dept_id IN (10,20)

* Nested Queries (Subqueries)

- A query inside another query.
- SQL query embedded within another SQL Statement (the outer query)

Key Concepts

1. Enclosed in Parentheses:

Subqueries must always be enclosed within parentheses ().

2. Location:

They can be placed in various clauses of the outer query including: SELECT, FROM, WHERE and HAVING.

Example

```
SELECT emp_name
FROM Employee
WHERE Salary >
(SELECT AVG(salary) FROM Employee);
```


* PL / SQL :

PL/SQL (Procedural Language / SQL) is an extension of SQL used for procedural programming.

Features :-

1. Variables
2. Loops
3. Conditions
4. Exception handling

Syntax :-

DECLARE
 --- variable declaration

BEGIN
 --- executable statements

EXCEPTIONS
 --- error handling

END;
 /

↓ Stored procedures

- A Stored procedure is a named PL/SQL program stored permanently in the database that is used to perform a specific task.

* Why Stored procedure used?

1. Reusability - write once, use many times
2. Better performance - executed on the database server
3. Security - Users can execute procedures without direct table access
4. Reduced network traffic - multiple SQL statements run as one block

• Syntax:

```
CREATE OR REPLACE PROCEDURE procedure_name
(
  parameter_name datatype [IN | OUT | INOUT]
)
AS
BEGIN
  --- SQL/PL-SQL statements
END;
/
```


* Procedure with Parameter (PL/SQL)

A procedure with parameter is a stored procedure that accepts values from the user and/or return values while executing its logic.

• Three Types of Parameters

1. IN
2. OUT
3. IN OUT

1. IN Parameter :- Used to pass input values to the procedure.

2. OUT Parameter :- Used to return a value from the procedure.

3. IN OUT Parameter :- Used to accept input and return modified output.

* Functions

A Function returns a value. Using RETURN Statements.

Functions are mainly used for calculations and computation.

• Syntax :

```
CREATE OR REPLACE FUNCTION Function-name
(
  parameter-name datatype
)
RETURN return-datatype
AS
BEGIN
  ----- Statements.
  RETURN value ;
END ;
/
```

• Example

```
CREATE OR REPLACE FUNCTION get_bonus
(
  Salary NUMBER
)
RETURN NUMBER
AS
BEGIN
  RETURN salary * 0.10 ;
END ;
/
```


* Triggers

- Triggers execute automatically when an event occurs. (INSERT, UPDATE, DELETE)
- They can be defined:
 - BEFORE the event
 - AFTER the event

* Assertions

- Assertions are conditions that must always be true for the database

e.g. $\text{Salary} > 0$

(conceptual topic, rarely implemented directly in SQL)

* Roles

Roles are collections of privileges.

example:

```
CREATE ROLE manager;
GRANT SELECT, INSERT ON EMPLOYEE
  TO manager;
```


* Privileges

Privileges define what actions a user can perform.

Types :

1. System privileges (CREATE TABLE)
2. Object privileges (SELECT, INSERT)